COMPX551-26B Assignment 4

Generative AI Declaration — Group 20

Each member of the team declared their own use of generative AI.

Contents

1 Jesse Du Plessis — Sensing and service **2**

2 Hamish Gilmore — Signal processing and calibration **7**

3 Mohammad Emon — Notification control **9**

4 Richard Larsen — UI, onboarding and stats; design and architecture **11**

1 Jesse Du Plessis — Sensing and service

AI declaration

Generative AI was used to generate code and aid in development in this assignment. AI was used to assist in generating code, debugging, modifying code, implementing functionality, improving responsive design, generating images for background and logo and generating graphs.

I provided my own prompts and reviewed all generated outputs to ensure correct functionality and understanding before incorporating into my assignment. AI output was tested and modified in Android Studio where necessary to meet requirements of the assignment and my intended design. I am responsible for understanding the submitted work and final implementation and design decisions.

The following prompts and descriptions show the most significant ways generative AI was used.

Part A — connecting to the Muse 2 and getting raw EEG data

1. *Can you add a lib file for a SDK library and a jniLibs as well.* The AI responded by creating the correct folders, however I had to modify their location so the lib and jniLibs folders were in the main folder.
2. *Can you add Bluetooth permissions for older Android devices to the manifest.* The AI responded by adding permissions to the manifest.
3. *Can you modify my MuseEegSource to have the basic connection functionality* (copied and pasted code snippets from the SDK example code). The AI modified MuseEegSource to have a Muse connection setup similar to the example. I tested this by adding some logs showing scanning and connecting for the Muse headband, running the application and the Muse, and checking that the logs clearly showed the Muse (and its unique name) had connected and the device name was displayed in the log.

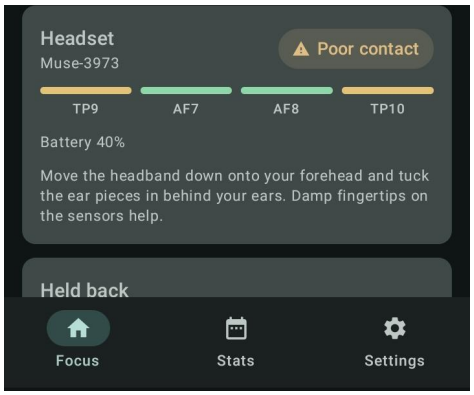


Figure 1: Part A: the headset connected, showing its name, battery and per-electrode contact.

Part B — FakeEeg setup

1. *Can you modify my fake EEG source code to give placeholder EEG readings, here's the example EEG connection setup* (copied and pasted some code snippets from the example code). The AI modified the functions in the file and added simulated data code for EEG readings. I tested the result by reading through it, making sure the original code's functions

remained and that only the simulated data was added and used instead of data from a connection to the Muse, then running the app to see if it went through the simulated flow and data. The app correctly showed fake data and cycled through focused and not focused states.

2. *Can you add some artifact functionality to the fake EEG source like how artifacts are shown with the Muse example code.* The AI generated some code and a function for random artifacts such as blinking to occur, for more realistic behaviour. I ran the application after adding some logs to see if artifacts were correctly appearing.

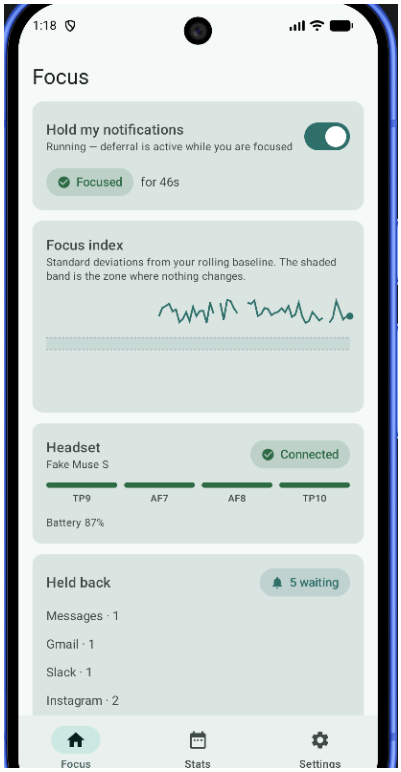


Figure 2: Part B: the app running on simulated EEG data.

Part C — MuseEegSource setup

1. *Can you modify my Muse EEG code to get the data packets* (copied and pasted some snippets of code from the Muse example code for data packets and some lines of text from the docs) *for the values listed here* (copied and pasted some text from the design document of what values the calibration section needs). The AI modified the code and added some functions to receive data and pass it through, and added methods for artifacts. I read through what the AI changed and added and compared it to the example code and docs. I had to make changes to the artifact functionality to fit the docs: the AI did not have the structure for getting blinks and jaw clenches, so it implemented them incorrectly and they needed some small edits. I then ran the application to check with logs that the Muse connected and was receiving the different values.

2. *Can you extend the delay? It does not allow enough time to connect.* The AI increased the delay and changed the functionality so that the application keeps scanning until a connection is made, instead of stopping the scan before the device can connect. I ran the application to test that the connection was more reliable.

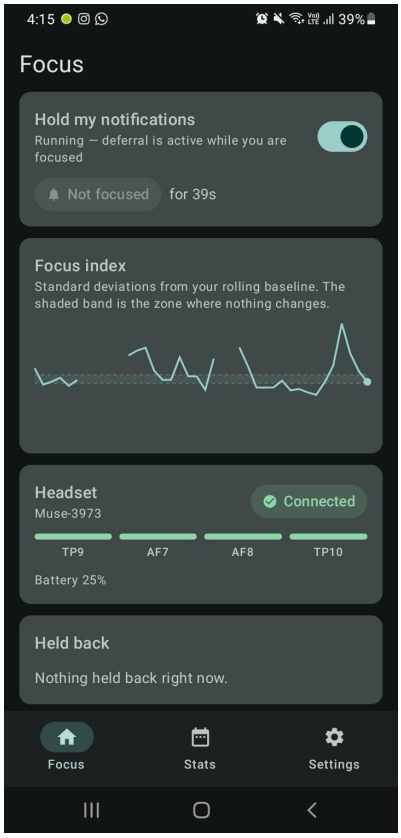


Figure 3: Part C: live data from the Muse through MuseEegSource.

Part D — FocusService setup

1. *Can you modify the focus service code? I need a foreground service to set up or handle everything while there is no active screen for the application. The epochs should be passed to the focus engine (workload B from the workload document) and pass the focus states to C's workload.* The AI modified the focus service and generated code. I tested the resulting code by running the application to check that it was still functioning without being the active screen. When the functionality was confirmed I went through the code and compared it to online sources on foreground services.

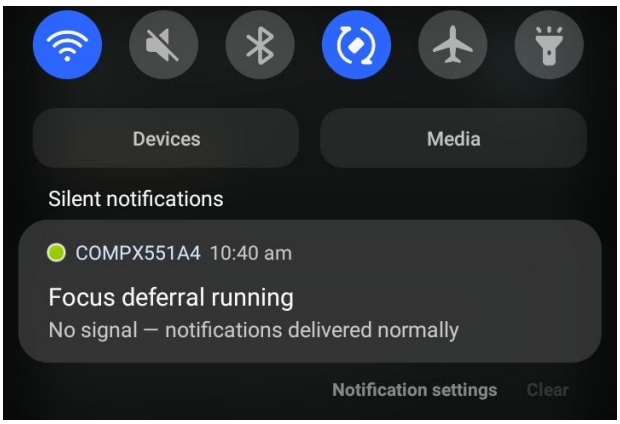


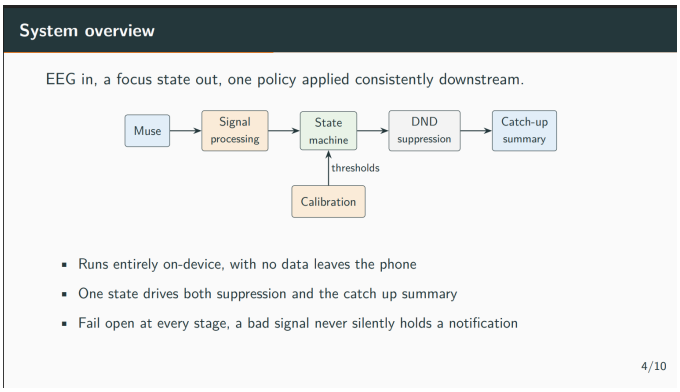
Figure 4: Part D: the foreground service's persistent notification.

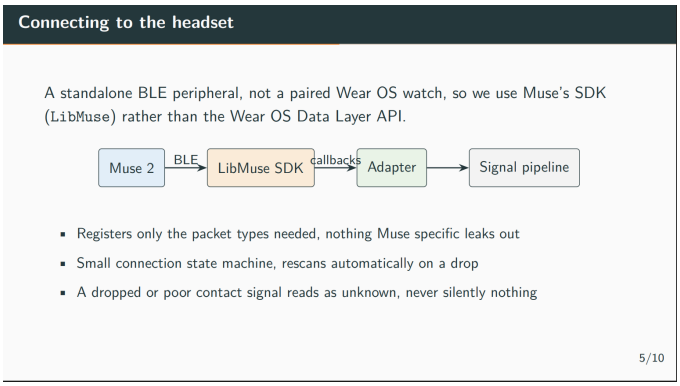
Part E — presentation slides

1. *Can you convert these paragraphs into LaTeX slides* (a PDF from Google Docs; the paragraphs are reproduced below). The AI responded with a LaTeX script with the contents of the PDF. I ran the LaTeX in Overleaf to view the slides.

2. *Can you give me a LaTeX script for a diagram of the following: Muse 2 headband connecting to the application and processing the signal, and calibration, to the focus state machine, then to the notification suppressing and the catch-up summary.* The AI responded with a diagram for the slide.

3. *Great, can I get another two diagrams for slides 2 and 3: Muse to SDK to adapter to signal processing; the other diagram for the different interfaces, so sensing and services, calibration, notifications and user interface.* The AI responded with LaTeX code for the two diagrams; some small changes were made to the diagrams so they better fit the slides.





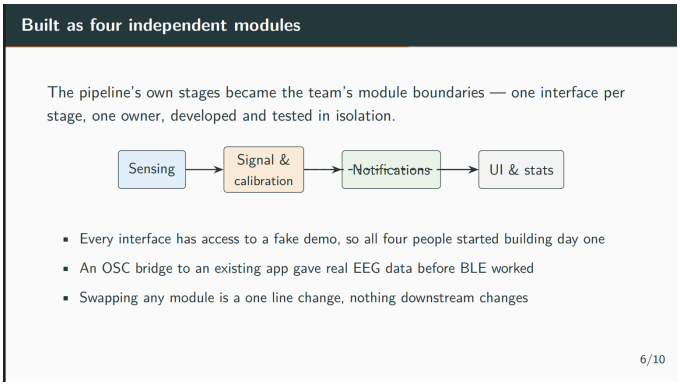


Figure 5: Part E: the three slides produced.

Original paragraphs for the slides

System overview. EEG data from the Muse is received and processed on the phone to determine a state of focused or not focused. The state determines if notifications are suppressed and the catch-up summary. Calibration is done separately to come up with thresholds for determining focus. A dropped connection or poor contact is treated as unknown so that we don't block notifications.

Connecting to the Muse. The Muse is a device that can be connected to with Bluetooth. We connect to the device using the LibMuse SDK instead of Wear OS. We have an adapter between the SDK and our signal processing so the rest of the app doesn't rely on the Muse functionality. There is also auto-reconnecting if the Muse disconnects, and missing data and poor contact are still kept as a state instead of being ignored.

Independent sections. Our system consisted of four separate sections: sensing, calibration and processing, notifications, and the UI. Each section uses its own interfaces, which allows our team to develop and test our own components independently using simulated data. We also explored an OSC sender to obtain real EEG data if an SDK or Bluetooth connection could not be used (the OSC sender was not needed in our project as we established the Bluetooth connection quite early on).

2 Hamish Gilmore — Signal processing and calibration

AI prompts

Before I began, the file structure had already been set up with stand-in versions of each part of the app's pipeline. A document detailing the workload split, and what each file should achieve, was also available to the LLM that I used (Claude). I didn't write or edit any code myself. The initial implementation of each component was written by Claude, as well as any subsequent changes such as diagnosing and fixing bugs or specific changes to things like window sizes. The main prompts I used are included below (formatted for the purpose of this document).

- Can you explain the project as it currently exists, and give me an overview of how each part interacts.
- I am person B. Can you give me an overview of what I need to implement, and suggest an appropriate order of implementation.
- Can you complete the implementation for the FocusStateMachine?
  - It should enter the FOCUSED state whenever the condition "index $> T_{\text{high}}$" is sustained for N seconds or more.
  - It should enter the NOT FOCUSED state whenever the condition "index $< T_{\text{low}}$" is sustained for M seconds or more.
- Now implement the ArtifactRejector. It should reject:
  - anything the source marked as invalid,
  - any electrode with worse than good contact,
  - non-finite values,
  - muscle noise judged against the user's own recent median (beta above 1.3×).

  After any artefact is rejected, it should also reject the next 2 s.
- Now the FocusPipeline, which should implement the FocusEngine interface. It should compute rolling z-scores of the Pope engagement index.
- Now implement the ThresholdStore which stores the threshold ratios and the calibration history.
- Implement the ThresholdFitter.
  - It should decide on the high and low thresholds for the focus index.
  - Include some hysteresis, so that the low threshold is lower than the high threshold.
  - It should also report AUC for the evaluation.
- Implement CalibrationSession.
  - It should drop the first few seconds of each block to account for settling in time.
  - It should use TaskScore to drop blocks where the user clearly didn't engage.

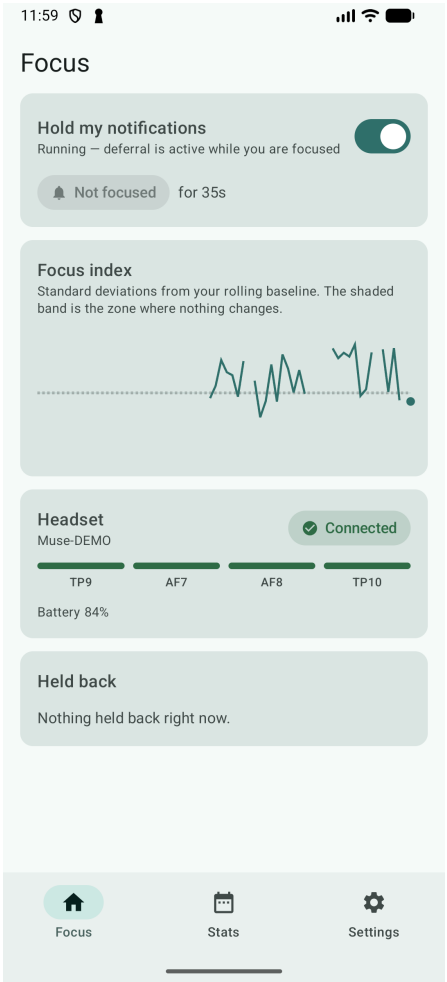
Code evaluation

Code was evaluated in the following ways:

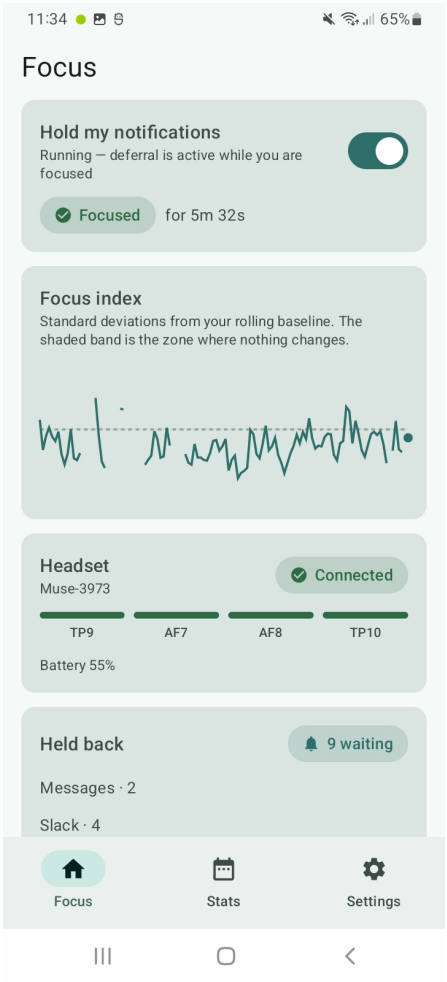
- Running the app to see if it was behaving as expected, and if there were any bugs.
- Looking at the code and checking that (for example) the z-score is calculated correctly, data was being sent and received from the right place, and so on.
- Looking up documentation for things I wasn't familiar with (e.g. libraries, structures related to lifecycle etc.) to make sure that it made sense in the context of the project.

Screenshots

Below are screenshots of the app running with fake data and with real data from the Muse headband.



Fake data



Real data

3 Mohammad Emon — Notification control

Prompts

1. *Can you explain how the shared Suppressor interface should be used for the notification-control workload, and how NotificationController should implement it?*

   AI explained how the Suppressor interface provides `apply()`, `flush()`, and the deferred notification StateFlow. I used this to structure the NotificationController so that it receives the focus state from the Focus Engine and controls notification suppression.

2. *Can you help me implement Android Do Not Disturb using AutomaticZenRule so that notifications can be suppressed when the user is focused?*

   AI provided an implementation using NotificationManager and AutomaticZenRule. I reviewed the code and adapted it to our project's FocusState and notification architecture.

3. *Can you implement a notification queue that stores the shared DeferredNotification objects while the user is focused?*

   AI helped implement the DeferralQueue. I changed it to use the shared DeferredNotification model from the core package rather than creating a separate notification model.

4. *Can you modify the NotificationListenerService so that incoming notifications are passed to the notification controller while focus mode is active?*

   AI helped modify FocusNotificationListener to convert Android notifications into DeferredNotification objects and pass them to the notification-control component. I tested the listener using Android notification test commands and Logcat.

5. *Can you implement a catch-up notification that groups all deferred notifications into one summary when focus ends?*

   AI helped implement the grouped catch-up notification. Instead of releasing all notifications individually, the application creates one summary notification showing the notifications that were received during the focused period.

6. *Can you modify the notification summary so that it can count different notification types such as calls, messages, emails and other notifications?*

   AI helped add notification category information to DeferredNotification and use it when creating the catch-up summary.

7. *Can you review the notification-control implementation and make sure it fails open when the focus state is UNKNOWN or the notification policy cannot be applied?*

   AI helped identify the fail-open behaviour required by the design. I reviewed and tested the implementation so that notifications are not silently blocked when the headset, signal, or notification-policy access is unavailable.

8. *Can you review my notification-control implementation and check that it connects correctly with the rest of the team's shared architecture?*

   AI helped identify integration issues and explain how the notification controller should be shared between the notification listener and the service. The final implementation was then integrated and tested on the main branch.

AI contribution and validation

Generative AI was used to assist with implementing, debugging, and reviewing the notification-control functionality. I provided the prompts, reviewed the generated code, and adapted it to the team's shared interfaces and architecture. The generated code was built and tested in Android Studio, and changes were made where the generated implementation did not match the project's requirements or structure. I am responsible for understanding the final implementation and the design decisions used in Part C.

4 Richard Larsen — UI, onboarding and stats; design and architecture

Overall use

I disclose that I used LLMs for all coding tasks. None of the code in my section was written by hand and my job was that of designer.

Design and architecture

Initial prompt

> We've previously talked about using the Muse headset for a project. The project we've decided on is: Using Muse with an android application to defer notifications until the user is no longer concentrating on a task. The basic idea is that the headset is reading the user's brain activity while they work, the app decides from that whether they're in a focused state or not, and while they are, incoming notifications get held back. Once the app decides they've come out of focus the held notifications get released all at once, or possibly in a short burst so the user isn't hit with twenty at a time. We have access to the Muse sdk, so getting the raw EEG and the band powers (alpha, beta, theta etc.) shouldn't be the hard part, the hard part is going to be deciding what "concentrating" looks like for a particular person and building the app around that.
>
> I imagine the application would need some features in order to make it user specific and actually usable day to day:
>
> The ability to turn on and off the notification deferral service. A simple toggle on the main screen, and probably a quick settings tile or a persistent notification with a pause button so the user doesn't have to open the app every time. It would also make sense for the service to turn itself off, or at least stop deferring, when the headset disconnects or is taken off, since at that point we have no data and shouldn't be holding anything back.
>
> The ability to run in the background so the user could be busy with something on their phone and still have the notifications deferred. From what I understand this means a foreground service on android to keep the bluetooth connection to the Muse alive, plus a notification listener service to actually intercept the notifications. I'm not sure how the deferral itself should work, whether we can genuinely hold a notification and re-post it later, or whether we need to cancel it, store the content, and recreate it ourselves when the user is free. Either way the user should be able to see what has been held so far if they really want to, and there should be some kind of whitelist so calls, alarms and selected contacts or apps always get through regardless of focus state.
>
> A feature that would make the thresholds user-specific. I was thinking of a brief benchmarking type activity where the user holds their finger on the screen, reads some text, releases when they are done and is instructed to 'relax' for a time period, then repeat this several times to gather enough information to create the user-specific thresholds. The finger on the screen is there so we have a clean label for which samples are "focused" and which are "relaxed" without the user having to do anything extra. Maybe five or six rounds of reading plus relaxing, so somewhere around five to ten minutes total. The reading material could be short passages of something fairly dense so they actually have to pay attention, and the relax phase could just be a blank screen with a countdown, or a breathing animation. Open to other ideas about how to do this. I've also wondered whether we need a third category, like a "distracted" or "idle phone use" state, where the user is scrolling but not actually working, since that's probably what the EEG looks like most of the time and I don't know how different it is from concentrating.
>
> Thresholds should be saved, per user, and the user should be able to re-run the calibration whenever they want, since I'd expect things to drift with headset placement, time of day, caffeine, and so on. It might be worth storing the last few calibrations and either averaging them or letting the user pick which one to use. There should probably also be a manual

fine-tune, like a sensitivity slider, so that if the app keeps thinking they're focused when they're not (or the other way round) they can nudge it without doing the whole calibration again.

There should probably be a time-out period so a brief lapse in concentration doesn't let the notifications through, this should be user-adjustable. Something like: the app only switches to "not focused" after the signal has been below the threshold for N seconds, with N defaulting to maybe thirty seconds or a minute and adjustable from a few seconds up to several minutes. It would allow for brief pauses in work without distractions, like looking out the window or reaching for a coffee. Similarly there probably needs to be a short delay before switching into the focused state, otherwise the first moments of sitting down would count as focus and start holding things straight away. I'm also wondering about a hard cap, so that nothing gets deferred for longer than, say, two hours no matter what the headset says, as a safety net against the app misreading someone and sitting on an important message all afternoon.

Beyond that I'd like some kind of log or history screen so the user can see when the app thought they were focused and how many notifications it held during each session. Partly for their own interest, partly so we can tell during development whether the thresholds are doing anything sensible. Signal quality from the Muse also needs handling somehow, if the electrodes aren't making good contact we should probably fall back to letting notifications through rather than guessing.

Things I'm less sure about: how much battery the constant bluetooth stream plus a foreground service is going to cost on the phone, and how long the Muse itself lasts in continuous use. Whether android's battery optimisation is going to kill the service after a while and how much the user will have to fiddle with settings to prevent that. Whether a single alpha/beta style ratio is actually going to separate focused from relaxed reliably enough for one person, or whether we'd need something more involved, and whether reading on a screen during calibration produces the same pattern as the actual work the user does (which might be typing, or talking, or working on paper, where eye and jaw movement artifacts are different). And whether the whole thing is annoying in practice, because being told your messages were held for you for forty minutes might feel worse than just getting them.

Can you think of any other ideas, or any problems with the ideas as proposed? I'm also interested in whether the calibration approach sounds reasonable or whether there's a better known way of doing this with consumer EEG, and roughly how you'd structure the android side so the headset handling, the focus detection and the notification handling stay separate enough that we can swap pieces out as we learn more.

UI

Once the design document was in place I was able to prompt the LLM to complete the UI section based on the contents of `DESIGN.md`.

Testing and verification

Several hours of prompting to fix UI quirks, some minor bugs, and having the LLM monitor data from the live application running while connected to the Muse to ensure there were no issues. This was many smaller prompts, which I won't list here.